Template do README do seu projeto

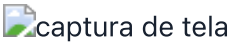
Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Copie o conteúdo abaixo da linha para o `README.md` do **seu** repositório e preencha. Apague as instruções em itálico.

📋 Nome do Projeto

Uma frase dizendo o que o projeto faz. Ex.: "Aplicação para registrar e acompanhar gastos mensais por categoria."

Uma segunda frase sobre o problema que ele resolve. Ex.: "Nasceu da dificuldade de saber para onde vai o dinheiro do mês sem precisar montar planilha."

 captura de tela

Tire um print da aplicação rodando com dados de verdade e coloque na raiz do repositório. Um projeto sem imagem no README perde metade do impacto.

🛠️ Tecnologias

- **Back-end:** Node.js, Express, SQLite (`node:sqlite`)
- **Front-end:** React, Vite
- *acrescente o que mais usou*

🚀 Como rodar

Pré-requisito: Node.js 22.5 ou superior.

Este projeto tem duas partes que rodam ao mesmo tempo. Abra **dois terminais**.

1. Back-end

```
cd backend
npm install
cp .env.example .env
npm run popular      # cria o banco e insere dados de exemplo
npm run dev
```

A API sobe em http://localhost:3000.

2. Front-end

```
cd frontend
npm install
cp .env.example .env
npm run dev
```

A interface abre em http://localhost:5173.

🔌 Rotas da API

Método	Rota	Descrição	Status
`GET`	`/recursos`	Lista tudo. Aceita `?filtro=valor`	200
`GET`	`/recursos/:id`	Busca um	200 / 404
`POST`	`/recursos`	Cria	201 / 400 / 409
`PATCH`	`/recursos/:id`	Atualiza em parte	200 / 400 / 404
`DELETE`	`/recursos/:id`	Remove	204 / 404

Exemplo de requisição

```
POST http://localhost:3000/recursos
Content-Type: application/json

{ "nome": "Exemplo", "categoria": "geral" }
```

Exemplo de resposta

```
{ "id": 1, "nome": "Exemplo", "categoria": "geral", "criadoEm": "2026-06-01T10:00:00.000Z"
        }
```

🗄️ Modelo de dados

```sql
CREATE TABLE recursos (
  id        INTEGER PRIMARY KEY AUTOINCREMENT,
  nome      TEXT    NOT NULL,
  categoria TEXT    NOT NULL DEFAULT 'geral',
  criado_em TEXT    NOT NULL
);
```

📐 Regras de negócio

Liste as regras que o projeto aplica — é aqui que se vê que existe pensamento por trás do CRUD.

- Não é possível cadastrar dois recursos com o mesmo nome (responde `409`)
- O valor precisa ser maior que zero
- ...

📁 Estrutura

```
projeto/
├── backend/
│   └── src/
│       ├── routes/          declara as rotas
│       ├── controllers/     traduz HTTP ↔ negócio
│       ├── services/        regras de negócio
│       ├── repositories/    acesso ao banco
│       ├── validacoes/
│       ├── middlewares/
│       └── banco/           schema, conexão e seed
└── frontend/
    └── src/
        ├── components/
        └── services/api.js  todas as chamadas HTTP
```

🤔 Decisões técnicas

Escreva 2 ou 3 parágrafos curtos. Esta seção é a que mais impressiona quem lê — e é a que o professor vai usar como base para as perguntas na apresentação.

Por que camadas? *Ex.: "Separei em routes/controller/service/repository porque na aula 07 trocamos o array por SQLite e só o repository precisou mudar. Preferi manter a estrutura mesmo sendo um projeto pequeno, porque..."*

Por que [alguma escolha sua]? ...

🚧 O que eu faria com mais tempo

- ...
- ...

👤 Autor

Seu Nome — @seuusuario (https://github.com/seuusuario)

Projeto desenvolvido no Módulo 1 do Curso de Formação de Programação Aplicada — FPA Labs.
